

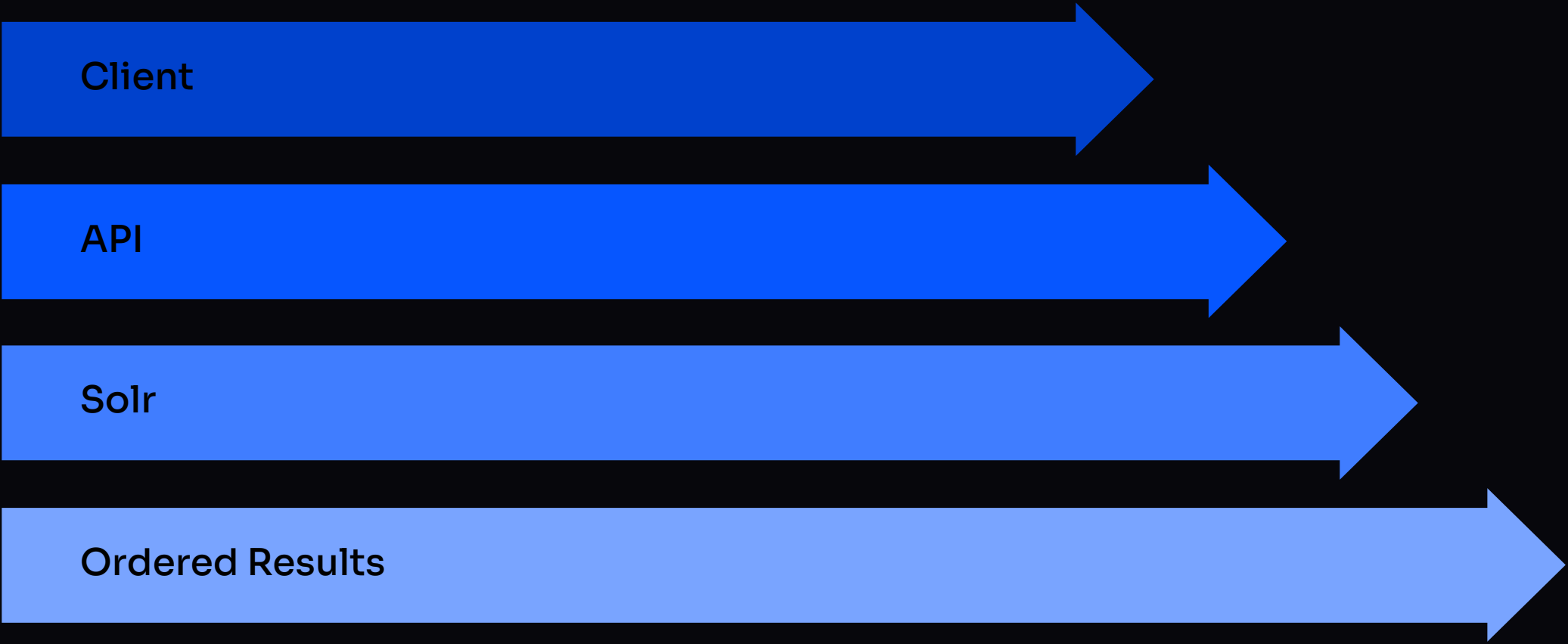
ARQUITECTURA DE SOFTWARE

Clase Práctica



Motores de búsqueda con Apache Solr

De Information Retrieval a una Search API en Go



Client

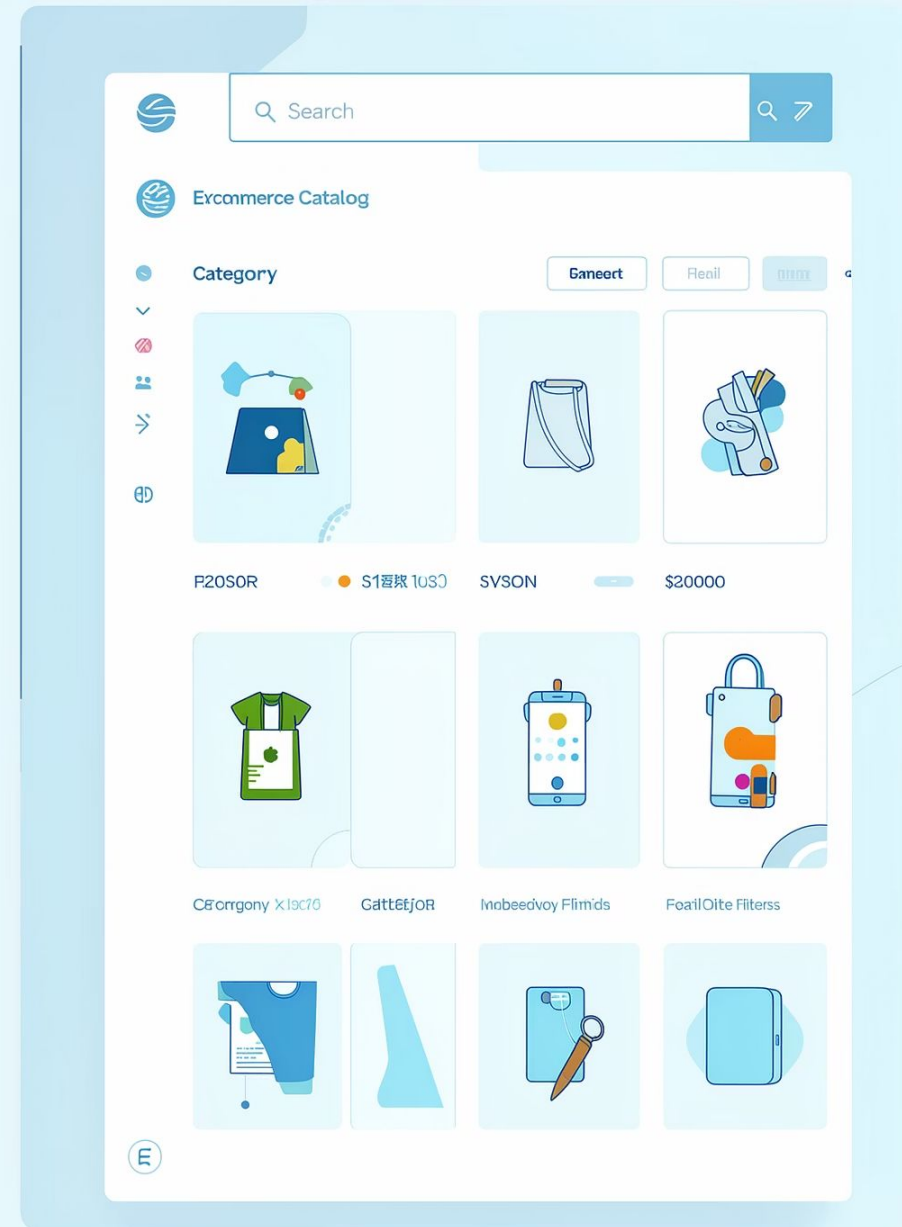
API

Solr

Ordered Results

El problema: buscar entre millones de productos

- Tenemos un e-commerce con catálogo de cientos de miles o millones de productos
- Cada producto: id, title, description, category, brand, price
- Usuarios escriben consultas libres:
 - zapatillas
 - zapatillas running
 - adidas running
 - calzado para correr
- Necesitamos encontrar rápidamente lo que buscan y ordenar por relevancia



"¿Qué esperamos de una buena búsqueda?"

- Encontrar rápidamente productos relacionados
- Permitir aplicar filtros (categoría, marca, precio)
- Ordenar resultados según relevancia

"¿Cómo encontramos lo que el usuario realmente busca entre millones de documentos y cómo decidimos qué resultado mostrar primero?"

Ejemplo de e-commerce:

- Usuario busca: "zapatillas"
- Usuario busca: "zapatillas running"
- Usuario busca: "calzado para correr"

"¿Alcanza con hacer un SELECT/LIKE sobre millones de productos?"

Necesitamos:

- Texto libre
- Relevancia
- Ranking
- Filtros
- Escalabilidad

Base de datos vs Motor de búsqueda

Base de datos (SQL)

```
SELECT *  
FROM products  
WHERE title LIKE '%zapatillas%';
```

Características:

- Fuente de verdad
- Transacciones
- Consultas estructuradas
- Estado del negocio

Motor de búsqueda

Características:

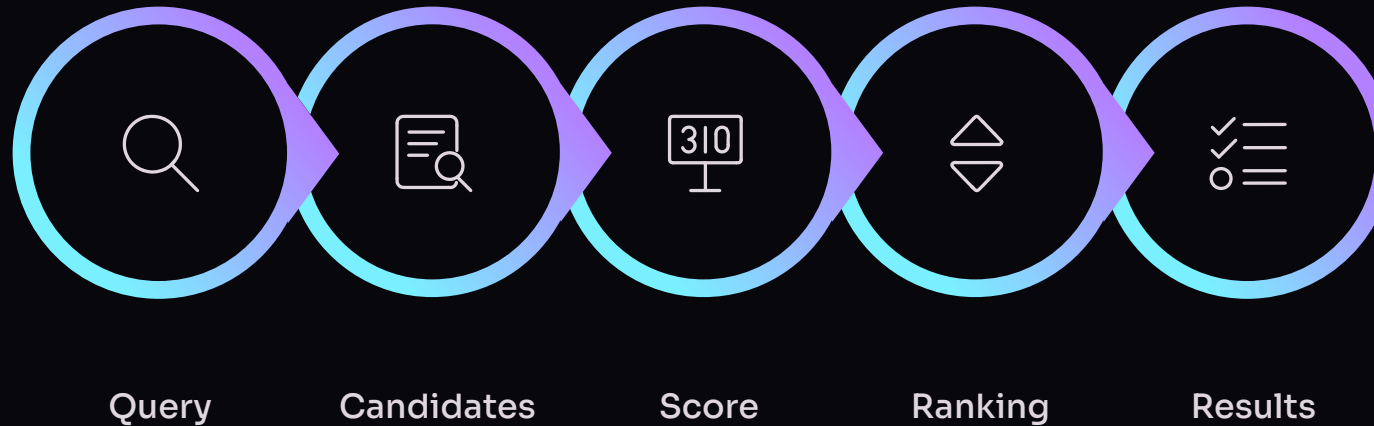
- Documentos
- Texto libre
- Análisis de texto
- Relevancia
- Ranking
- Filtros/facetado

❏ "Solr no reemplaza necesariamente a MongoDB/SQL: cumple otra responsabilidad."

Nota: Las bases de datos SQL modernas también ofrecen búsqueda full-text. Este ejemplo compara un LIKE sencillo contra un motor especializado. La diferencia está en capacidades de ranking, análisis de texto y escalabilidad.

Recuperación de información

Definición: "Recuperar los documentos más relevantes para una consulta."



Explicación de cada paso:

- **Query:** lo que el usuario busca
- **Candidatos:** documentos que coinciden
- **Score:** relevancia de cada candidato
- **Ranking:** ordenamiento por score
- **Resultados:** documentos ordenados al usuario

Ejemplo de e-commerce:

- **Query:** "zapatillas running"
- **Candidatos:** todos los productos que contienen esas palabras
- **Score:** cada producto recibe una puntuación de relevancia
- **Ranking:** se ordenan de mayor a menor score
- **Resultados:** se muestran al usuario en ese orden

Análisis de texto

"Proceso de transformación de texto para indexación y búsqueda."

Ejemplo concreto:

Texto original: "Zapatillas ADIDAS para Running!"

Proceso paso a paso:

1. Tokenización: ["Zapatillas", "ADIDAS", "para", "Running"]
2. Normalización (minúsculas): ["zapatillas", "adidas", "para", "running"]
3. Eliminación de stopwords (opcional, depende de configuración): ["zapatillas", "adidas", "running"]
4. Tokens finales indexados: zapatillas, adidas, running

Aclaración importante: "La eliminación de stopwords depende de la configuración del Analyzer en Solr. En este práctico, 'para' se elimina porque está en la lista de stopwords en español."

En Solr: "El Analyzer define este proceso durante indexación y búsqueda."

Ranking: ¿qué resultado aparece primero?

Pregunta central:

"¿Un match en title debería valer igual que un match en description?"

Ejemplo de comparación:

Query: "running"

Documento A:

- title = "Zapatillas Adidas Running"
- match en title
- score: 8.5

Documento B:

- description = "Ideal para running ocasional"
- match en description
- score: 4.2

Documento C:

- title = "Remera"
- description = "Perfecta para running"
- score: 3.1

Explicación:

"Ambos coinciden con 'running', pero tienen distinto score."

Conclusión:

"El motor calcula un score para cada documento y ordena por relevancia."

Configuración en Solr:

Para que title tenga más peso que description, usamos el parámetro qf (Query Fields) con boost:

`qf=title^5 description`

Esto significa: buscar en title con peso 5, y en description con peso 1 (por defecto).

De Product a documento Solr

Ejemplo de transformación

Mostrar el modelo de dominio (MongoDB):

```
{
  "id": "p1",
  "title": "Zapatillas Adidas Running",
  "description": "Calzado liviano para correr",
  "category": "calzado",
  "brand": "Adidas",
  "price": 120000
}
```

Clasificación de campos para Solr en tres columnas:

Campos de texto (indexados para búsqueda):

- title
- description

Campos de filtro (para restricciones):

- category
- brand

Campos numéricos:

- price

❏ **"No necesariamente se indexa todo el modelo de dominio. Se eligen los datos necesarios para búsqueda, filtrado y respuesta."**

Primera búsqueda: q, fq, defType y qf

Mostrar la URL base:

```
/solr/products/select?q=running
```

Explicar los parámetros principales:

q (Query)

"Consulta principal. Participa en la recuperación y ranking."

defType=edismax

"Define el tipo de parser. eDisMax es más flexible y permite boost de campos."

fq (Filter Query)

"Restringe resultados sin afectar el score. Ideal para categorías, marcas, rangos de precio."

qf (Query Fields)

"Especifica en qué campos buscar y con qué peso. title^5 significa title con peso 5."

Caso de uso:

"Buscar zapatillas running Adidas dentro de calzado."

Ejemplo de URL completa:

```
/solr/products/select?q=zapatillas running&fq=category:calzado&fq=brand:Adidas&defType=edismax&qf=title^5 description
```

Explicación:

- **q=zapatillas running** → busca estos términos y calcula relevancia
- **fq=category:calzado** → solo productos en categoría calzado
- **fq=brand:Adidas** → solo marca Adidas
- **defType=edismax** → usa eDisMax para una búsqueda más flexible
- **qf=title^5 description** → busca en **title** con mayor peso y también en **description**

Controlar la respuesta: rows, fl y score

Explicar tres parámetros clave:

rows

Cantidad máxima de resultados a devolver. Por defecto 10.

fl (Field List)

Campos que queremos devolver en la respuesta. Separados por comas.

score

Relevancia calculada por Solr. Aparece en la respuesta si está en fl.

Ejemplo completo:

```
/solr/products/select?q=running&fq=category:calzado&rows=5&fl=id,title,brand,price,score&defType=edismax&qf=title^5 description
```

Desglose:

- **q=running** → busca "running"
- **fq=category:calzado** → solo calzado
- **rows=5** → máximo 5 resultados
- **fl=id,title,brand,price,score** → devolver estos campos
- **defType=edismax** → usa eDisMax parser
- **qf=title^5 description** → busca en **title** con peso 5 y **description** con peso 1

Controlar la respuesta: rows, fl y score

Respuesta esperada:

```
{
  "response": {
    "numFound": 42,
    "docs": [
      {
        "id": "p1",
        "title": "Zapatillas Adidas Running",
        "brand": "Adidas",
        "price": 120000,
        "score": 8.5
      },
      ...
    ]
  }
}
```

❏ **Nota:** numFound indica cuántos documentos coinciden en total, pero rows limita cuántos se devuelven. Los números en este ejemplo (numFound: 42, score: 8.5) son ilustrativos. Con tu dataset real, los valores serán diferentes.

HASTA ACA LA TEORÍA

HANDS ON: levantar Solr

Realizar un Hands On guiado desde cero

Usaremos Docker para que todos tengan el mismo entorno.

1

Paso 1: Levantar el contenedor

```
docker compose up -d
```

2

Paso 2: Acceder a Solr Admin

<http://localhost:8983>

3

Paso 3: Verificar la collection

Buscar la collection 'products' en el panel de Solr Admin

4

Paso 4: Cargar dataset

El ZIP de la clase incluye docker-compose.yml y products.json

- ✓ Contenedor levantado
- ✓ Solr Admin accesible
- ✓ Collection 'products' visible
- ✓ Dataset listo para indexar

📌 **Nota importante:** "Todos deben tener Solr corriendo en localhost:8983 antes de continuar."

HANDS ON: indexar documentos en Solr

Ahora indexaremos los productos en la collection.

1

Paso 1: Crear el schema

Antes de indexar, definimos la estructura de los documentos. El archivo schema.json define los campos y sus tipos.

```
{
  "fields": [
    {"name": "id", "type": "string", "indexed": true, "stored": true},
    {"name": "title", "type": "text_es", "indexed": true, "stored": true},
    {"name": "description", "type": "text_es", "indexed": true, "stored": true},
    {"name": "category", "type": "string", "indexed": true, "stored": true},
    {"name": "brand", "type": "string", "indexed": true, "stored": true},
    {"name": "price", "type": "pint", "indexed": true, "stored": true}
  ]
}
```

```
curl -X POST http://localhost:8983/solr/products/schema \
-H "Content-Type: application/json" \
-d @schema.json
```

2

Paso 2: Preparar el dataset

El archivo products.json contiene documentos como:

```
{
  "id": "p1",
  "title": "Zapatillas Adidas Running",
  "description": "Calzado liviano para correr",
  "category": "calzado",
  "brand": "Adidas",
  "price": 120000
}
```

HANDS ON: indexar documentos en Solr

Ahora indexaremos los productos en la collection.

1

Paso 3: Indexar con POST

```
curl -X POST http://localhost:8983/solr/products/update \
-H "Content-Type: application/json" \
-d @products.json
```

2

Paso 4: Confirmar la indexación

```
curl http://localhost:8983/solr/products/update?commit=true
```

Explicación del proceso:

El Analyzer de Solr procesa cada documento: tokeniza, normaliza y crea el índice invertido.

Verificación:

En Solr Admin, la collection 'products' debe mostrar documentos indexados.

Pregunta:

¿Cuántos documentos se indexaron?

HANDS ON: primeras búsquedas

Ahora ejecutaremos búsquedas reales contra Solr.

Búsqueda 1: Todos los documentos

```
http://localhost:8983/solr/products/select?q=*:*
```

- Observar: numFound, documentos, estructura de respuesta

Búsqueda 2: Buscar "zapatillas"

```
http://localhost:8983/solr/products/select?q=zapatillas&defType=edismax&qf=title^5description
```

- Observar: cuántos documentos coinciden, cómo el qf prioriza title sobre description y cómo impacta el score

Búsqueda 3: Buscar "zapatillas AND running" con filtro de categoría

```
http://localhost:8983/solr/products/select?q=zapatillas AND running&fq=category:calzado&rows=5&fl=id,title,brand,price,score&defType=edismax&qf=title^5description
```

- numFound: total de coincidencias
- docs: documentos devueltos (máximo 5)
- score: relevancia de cada resultado
- qf: influye en la relevancia, dando más peso a title que a description
- orden: por score descendente

Nota importante:

- ❏ Nota: Un espacio entre términos no garantiza AND. Usamos 'zapatillas AND running' explícitamente. El comportamiento depende del parser (defType) y sus parámetros. Con eDisMax, el operador por defecto es OR.

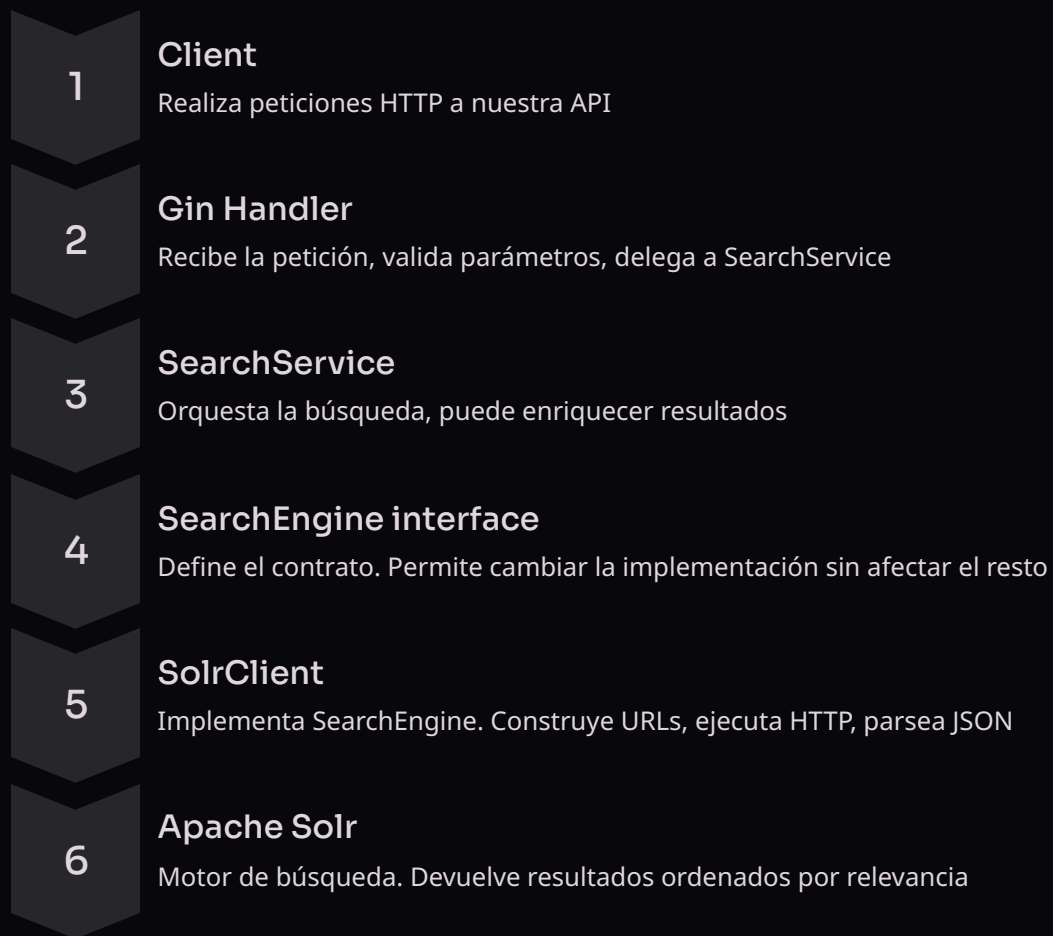
Pregunta clave:

"¿Por qué Solr encuentra estos documentos sin recorrer manualmente cada producto?"

Porque usa el índice invertido que creó durante la indexación.

Solr dentro de una API Go

Arquitectura desacoplada



📌 **Ventaja:** "MongoDB puede continuar como fuente de verdad. Solr devuelve IDs/scores y se enriquece desde Mongo si es necesario."

EJERCICIO 1: Search Lab en Solr

Dataset/collection 'products' ya disponible en Solr.

Resolver las siguientes consultas:

01	02	03
Buscar "zapatillas" Construir la URL y observar resultados	Buscar "running" Construir la URL y observar resultados Nota: Para buscar 'zapatillas' Y 'running' (ambos términos), usa: q=zapatillas AND running	Filtrar por category=calzado Agregar fq a la búsqueda anterior
04	05	06
Agregar filtro por brand Buscar "zapatillas" en category=calzado y brand=Adidas	Limitar a 5 resultados Usar rows=5	Devolver campos específicos fl=id,title,brand,category,price,score
07		

Comparar scores y explicar el orden

¿Por qué algunos documentos tienen mayor score que otros?

Desafío (bonus):

"Hacer que una coincidencia en 'title' tenga más peso que una coincidencia solamente en 'description'."

Objetivo:

"Entender q, fq, rows, fl, score y relevancia antes de integrar código."

Preguntas de reflexión:

- ¿Cuántos documentos coinciden con 'zapatillas'?
- ¿Cuántos coinciden con 'zapatillas AND running' (ambos términos)?
- ¿Cómo cambia el score si agregamos un filtro?

EJERCICIO 2: Go + Gin + Solr

Proyecto base incluido en el ZIP de la clase.

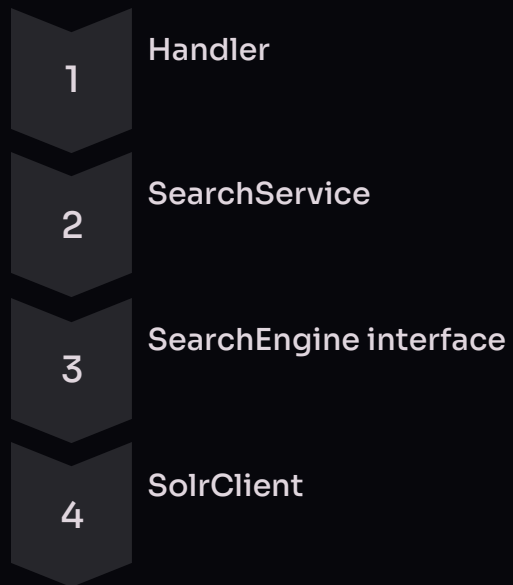
Implementar el endpoint:

```
GET /products/search
```

Parámetros:

- q (obligatorio): término de búsqueda
- category (opcional): filtro de categoría
- brand (opcional): filtro de marca
- limit (opcional): cantidad de resultados

Completar la arquitectura:



EJERCICIO 2: Go + Gin + Solr

El SolrClient debe:

01

Construir la URL base:
/solr/products/select

02

Transformar q (parámetro de entrada)

03

Transformar category/brand a fq (Filter Query)

04

Transformar limit a rows

05

Pedir los campos necesarios (fl)

06

Ejecutar HTTP GET contra Solr

07

Parsear JSON de respuesta

08

Mapear a SearchResult

09

Manejar errores (Solr no disponible, q vacío, etc.)

EJERCICIO 2: Go + Gin + Solr

Estructura esperada de respuesta:

```
{
  "results": [
    {
      "id": "p1",
      "title": "Zapatillas Adidas Running",
      "brand": "Adidas",
      "category": "calzado",
      "price": 120000,
      "score": 8.5
    }
  ],
  "total": 42
}
```

Criterios de éxito:

- q es obligatorio
- filtros se convierten a fq
- limit se convierte a rows
- error de Solr es controlado
- Handler no conoce detalles de Solr
- Solr está detrás de una interfaz

EJERCICIO 2: validación + cierre

Validar la implementación con estos curls:

Curl 1: Búsqueda simple

```
curl "localhost:8080/products/search?q=running"
```

Curl 2: Con filtro de categoría

```
curl "localhost:8080/products/search?q=running&category=calzado"
```

Curl 3: Con múltiples filtros

```
curl "localhost:8080/products/search?q=zapatillas&brand=Adidas&limit=5"
```

Contrato HTTP esperado:

```
{
  "results": [
    {
      "id": "p1",
      "title": "Zapatillas Adidas Running",
      "brand": "Adidas",
      "category": "calzado",
      "price": 120000,
      "score": 8.5
    }
  ],
  "total": 42
}
```



EJERCICIO 2: validación + cierre

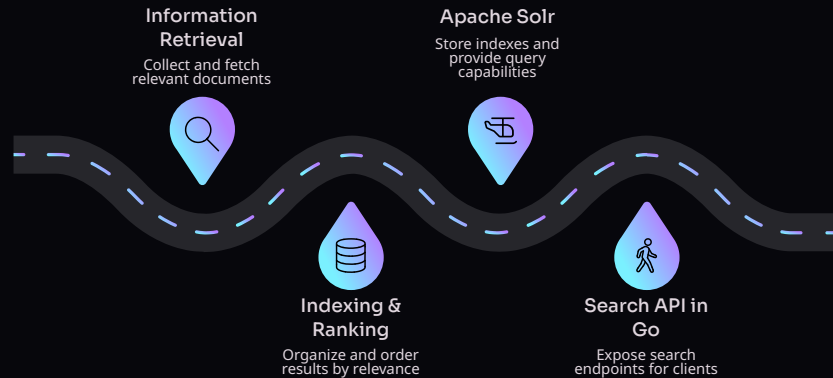
Criterios de validación:

- ✓ q es obligatorio (error si falta)
- ✓ filtros → fq en Solr
- ✓ limit → rows en Solr
- ✓ error de Solr controlado
- ✓ Handler no conoce detalles de Solr
- ✓ Solr está detrás de una interfaz

Bonus (si hay tiempo):

- Implementar cache-aside
- Agregar facetas por marca/categoría
- Boost de title sobre description
- Endpoint de reindexación

Cierre visual:



"Un sistema RAG puede utilizar un motor de búsqueda/retriever para recuperar contexto antes de llamar a un LLM."

Preguntas de arquitectura:

- ¿Por qué el Handler no conoce Solr?
- ¿Cómo conviven MongoDB y Solr?
- ¿Qué indexamos y qué no?
- ¿Qué ocurre con la consistencia cuando Mongo cambia antes que Solr?

